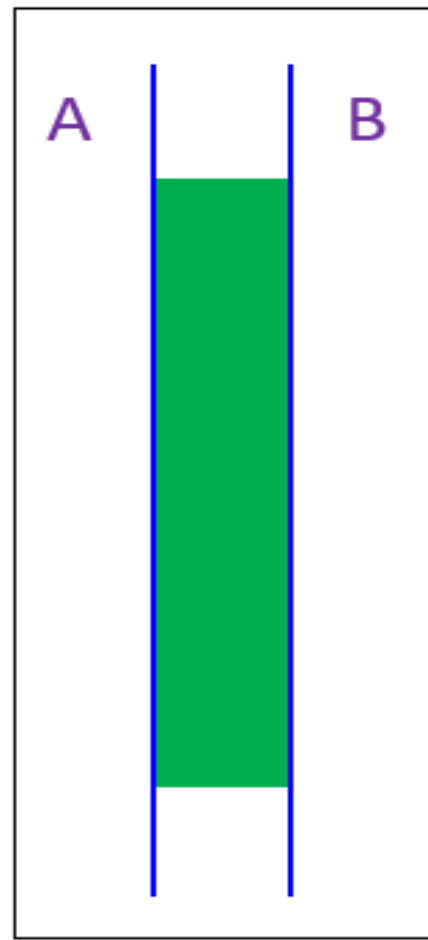
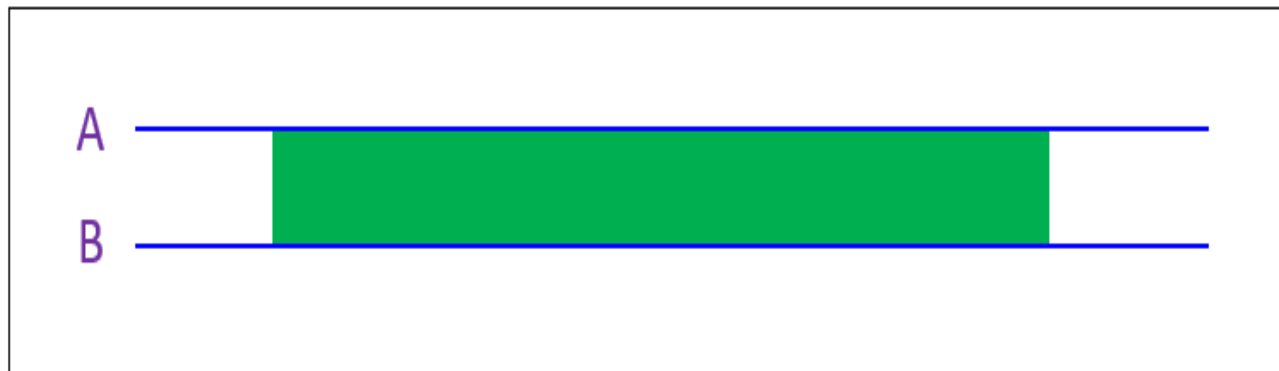


第13章

影像梯度與邊緣偵測

13-1：影像梯度的基礎觀念

- 13-1-1：直覺方法認識影像邊界
- 影像對於A與B線條而言，因為上方與下方的像素值差異不是0，所以可以知道A與B線條是邊界線條。但若上方像素與下方像素值相同，就可以知道不是邊界像素。

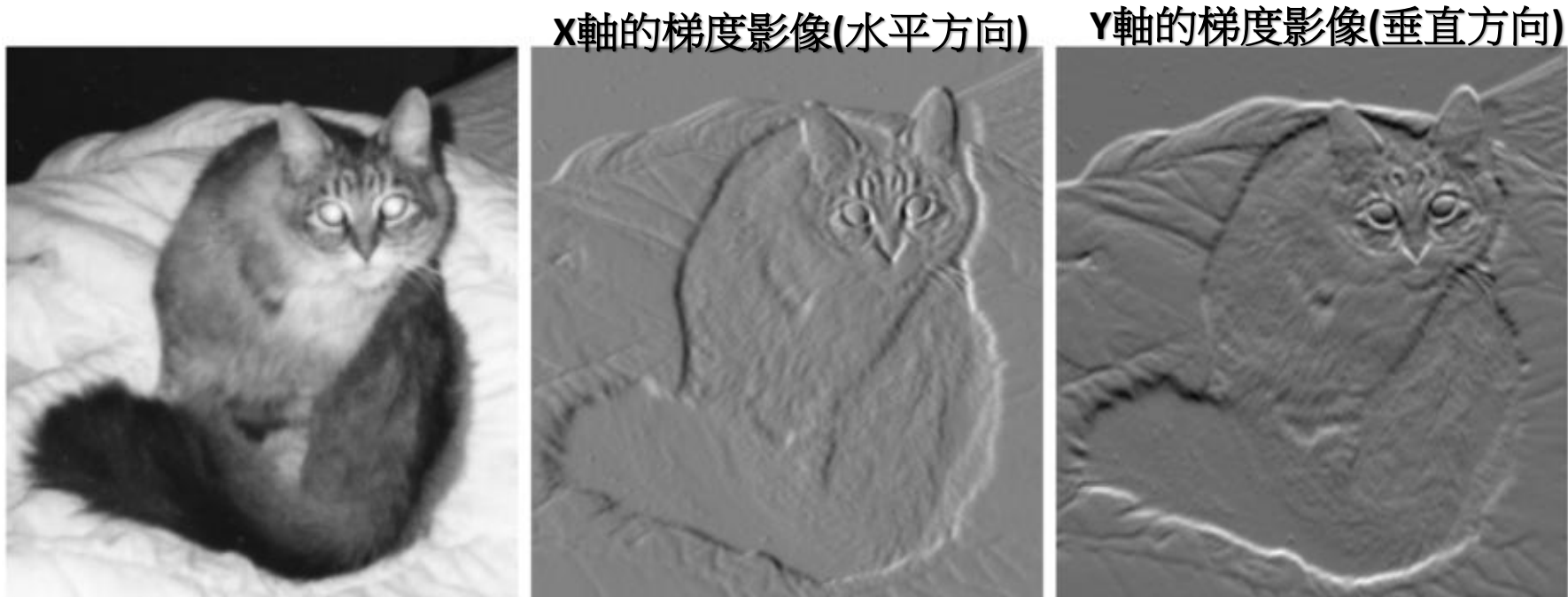


13-1-2：認識影像梯度

- 影像梯度：指影像強度與顏色方向的變化性，當影像的像素值變化較大時影像梯度的值也會比較大，這可能就是影像邊緣的位置。**光影變化在術語上就是所謂「梯度」**
- 所以可以使用影像梯度來計算影像的邊緣訊息。
- 如果將影像稱影像函數，則影像函數就是一個雙變數函數，兩個變數分別為X軸與Y軸。
- 影像函數的偏微分，可分為一個二維向量，分別代表X軸(橫軸)和Y軸(縱)
- 偏微分函數： $\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$
- $\frac{\partial f}{\partial x}$ 是X軸的梯度。 $\frac{\partial f}{\partial y}$ 是Y軸的梯度
- 常用的方法有Sobel, Scharr, Laplacian..等當作卷積核，執行卷積計算。

13-1-3：機器視覺

- 在機器視覺應用中，影像梯度可以抽取影像的資訊，最後產生一張梯度影像，主要是根據影像位置的特定方向變化計算出來的，也就是將X軸與Y軸的梯度與像素區塊做計算，最後再予以融合。
- 左邊為原始圖，中間為利用X軸計算出來的梯度影像，代表影像像素值在水平方向的強度變化。右邊是Y軸計算出來的梯度影像，代表影像像素值在垂直方向的強度變化。



- **Edge detection**邊緣偵測是**Computer vision**中最重要的一步驟，它讓電腦能準確的抓取圖素中的物體，邊緣指的是圖像中像素的值有突變的地方。
- **OpenCV**提供三種邊緣檢測方式來處理**Edge detection**：**Laplacian**、**Sobel**及**Canny**，這些技術皆是使用灰階的影像，基於每個像素灰度的不同，利用不同物體在其邊界處會有明顯的邊緣特徵來分辨
- 使用技術原理可分為兩種：
 - **Laplacian**原稱為**Laplacian method**，透過計算零交越點上光度的二階導數（detect zero crossings of the second derivative on intensity changes），**Laplacian**對於雜訊（**Noise**）非常敏感，因此在實用上都會將影像先模糊化後再處理(**LoG Laplacian of Gaussian**)。
 - **Sobel**使用的則是**Gradient methods**（梯度原理），透過計算像素光度的一階導數差異（detect changes in the first derivative of intensity）來進行邊緣檢測。**Sobel**以簡單的卷積過濾器（convolutional filter）偵測圖像上水平及縱向光度的改變，以加權平均方式計算各點的數值來決定邊緣。
 - **Canny**亦是利用梯度原理，**Canny**比起**Sobel**較能處理雜訊問題，但是需要花費較多的硬體資源來處理。它先將影像模糊化再進行非極大值抑制(non-maxima suppression)

13-2 : OpenCV函數Sobel()

- 13-2-1 : Sobel運算

- 濾波核概念
(遮罩Mask)
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

用於 x 軸算子

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

用於 y 軸算子

13-2-2：使用Sobel算子計算x軸方向影像梯度

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * A$$

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} p1 & p2 & p3 \\ p4 & p5 & p6 \\ p7 & p8 & p9 \end{bmatrix}$$

Mask

影像像素值

$$p5_x = (p3 - p1) + 2 * (p6 - p4) + (p9 - p7)$$

- 與雙邊濾波器觀念相近:除了使用像素之間空間鄰近度之外，還多考慮了像素之間的光度/色彩差異，達到保護邊緣與去噪的目的。考慮空間域核和值域核

- (1) 空間域核：由畫素位置歐幾里德距離決定, 計算臨近點到中心點臨近程度(二點之間的距離相減平方再開更號), 距離越遠權重越低
- (2) 值域核：由畫素值的差值決定, 兩點之間的畫素值相似程度，越相似權重越大
- (3) 將上述兩個相乘就得到了雙邊濾波器的模板權值

13-2-3：使用Sobel算子計算y軸方向影像梯度

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * A \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} p1 & p2 & p3 \\ p4 & p5 & p6 \\ p7 & p8 & p9 \end{bmatrix}$$

A為原始像素值

$$p5_y = (p7 - p1) + 2 * (p8 - p2) + (p9 - p3)$$

梯度計算 $G = \sqrt{G_x^2 + G_y^2}$ 或 $G = |G_x| + |G_y|$

PS.計算Gx 與 Gy, 其值大於255, 則取255

黃色網底: 原始像素值; 藍色網底:影像像素值邊界位置處理 紅色框線: Mask範圍, 像素值與Gx, Gy進行卷積計算
紫色框線:區塊核心像素值(濾波核核心)

步驟1: 先進行影像邊界處理 **步驟2:** 計算每個像素值賽Gx 與Gy 之和 **步驟3:** 設置門檻值 threshold, 來找出邊緣像素值

99	99	104	105	102	99	104	104
99	99	104	105	102	99	104	104
89	89	86	86	83	88	87	87
85	85	81	81	84	67	83	83
88	88	88	88	86	88	88	88
186	186	185	187	179	187	186	186
157	157	152	155	160	157	151	151
157	157	152	155	160	157	151	151

99	99	104	105	102	99	104	104
99	99	104	105	102	99	104	104
89	89	86	86	83	88	87	87
85	85	81	81	84	67	83	83
88	88	88	88	86	88	88	88
186	186	185	187	179	187	186	186
157	157	152	155	160	157	151	151
157	157	152	155	160	157	151	151

Gx(0,0)=

-1	0	1
-2	0	2
-1	0	1

X

99	99	104
99	99	104
89	89	86

+

Gy(0,0)=

-1	-2	-1
0	0	0
1	2	1

X

99	99	104
99	99	104
89	89	86

先乘後加總

練習1: Sobel

- 讀取LENA.JPG

13-2-4 : Sobel()函數

- `dst = cv2.Sobel(src, ddepth, dx, dy, ksize, scale, delta, borderType)`
- `ddepth`: 影像深度, 為-1:代表與原始影像相同。結果影像深度必須大於等於原始影像。
- `dx`: x軸的求導階數
- `dy`: y軸的求導階數
- `ksize`: sobel算子的大小, 可為1,3,5....
- `scale`: 計算導數的縮放係數, 預設為1
- `delta`: 可加到dst的值, 預設為0

- **Ddepth:** 使用多少個位元數來儲存像素值, 即為影像深度, 灰階影像為8位元。彩像影像

輸出目前影像深度要大於等於原始影像

來源影像深度	目標影像深度	值
cv2.CV_8U	cv2.CV_16S cv2.CV_32F cv2.CV_64F	8-bit unsigned integers (0..255)
cv2.CV_16U	cv2.CV_32F	16-bit unsigned integers (0..65535)
cv2.CV_16S	cv2.CV_64F	16-bit signed integers (-32768..32767)
cv2.CV_32F	cv2.CV_32F cv2.CV_64F	32-bit floating-point numbers (-FLT_MAX..FLT_MAX, INF, NAN)
cv2.CV_64F	cv2.CV_64F	64-bit floating-point numbers (-DBL_MAX..DBL_MAX, INF, NAN)

PS. Sobel 運算設定 ddepth :

主要是因為計算導數（微分）會產生正負數值變化，甚至可能超出原本影像像素範圍，例如：

- 原圖是8-bit（0~255），微分後可能是 -255 到 +255。
- 因此，輸出需要更大的範圍或有符號型態來正確儲存結果，避免資料截斷或溢位。

13-2-5：考量ddepth與取絕對值函數 `convertScaleAbs()`

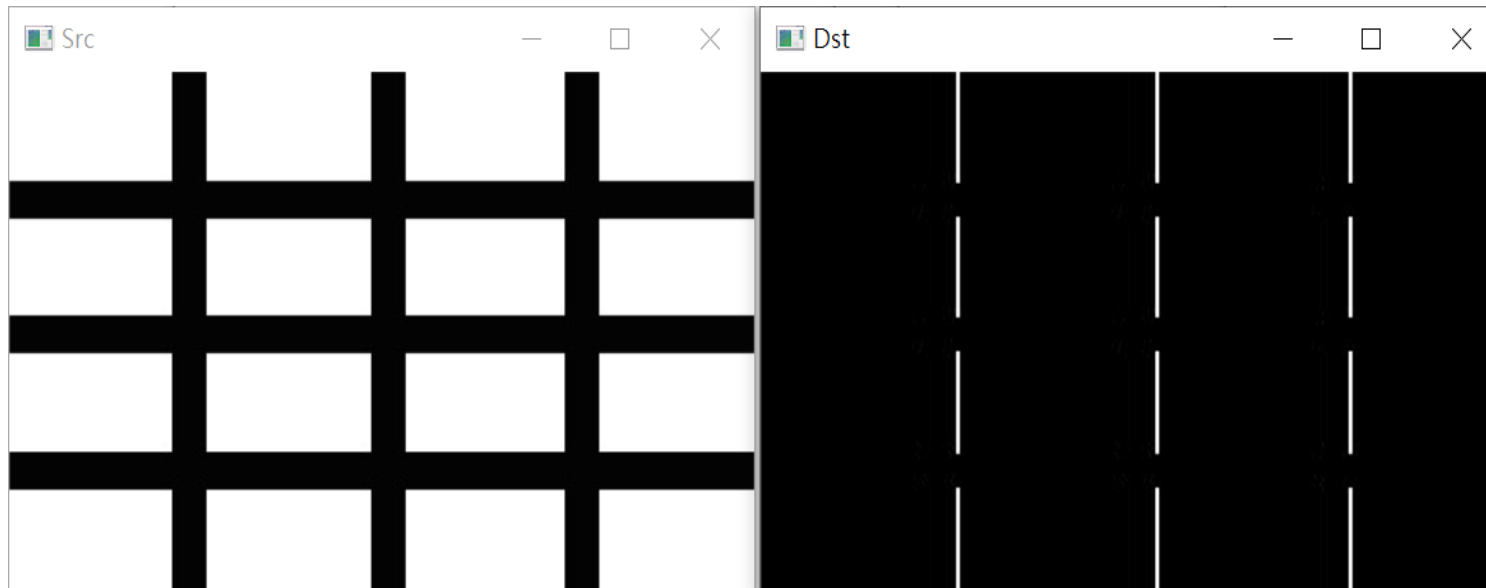
- 在計算影像梯度值時, 可能得到負數結果, 所以需要轉成正數
- `dst = cv2.convertScaleAbs(src, alpha, beta)`
- 程式實例1.py：使用`convertScaleAbs()`函數將一個含負數的矩陣，全部轉為正值。

```
===== RESTART: D:/OpenCV_Python/ch13/ch13_1.py =====  
src =  
[[ -53 -175  -96  130 -100]  
 [  48   5 -223   33   57]  
 [  17 -244  137  -51 -142]]  
dst =  
[[ 53 175  96 130 100]  
 [ 48  5 223  33  57]  
 [ 17 244 137  51 142]]
```

- `import cv2`
- `import numpy as np`
- `src = np.random.randint(-256,256,size=[3,5],dtype=np.int16)`
- `print(f"src = \n {src}")`
- `dst = cv2.convertScaleAbs(src)`
- `print(f"dst = \n {dst}")`

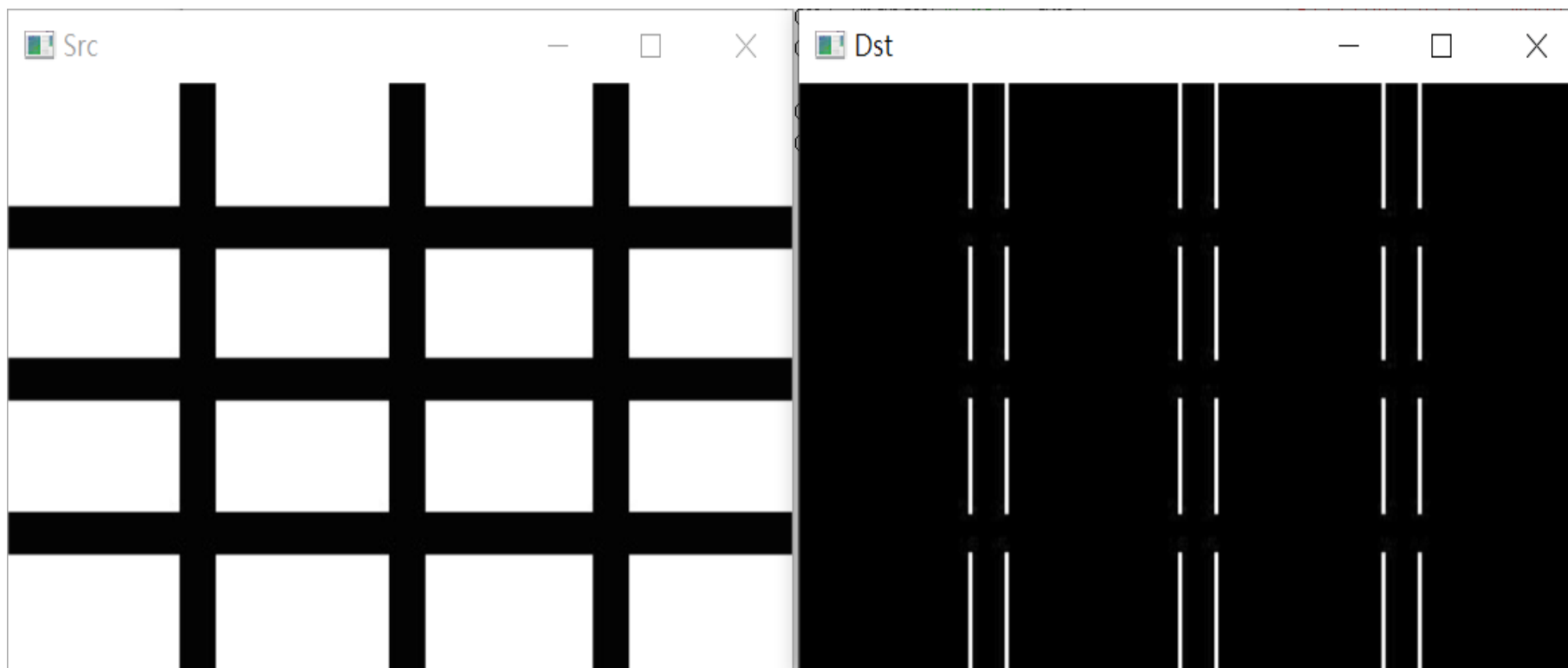
13-2-6：x軸方向的影像梯度

- `dst = cv2.Sobel(src, ddepth, 1, 0)` # 設定0,表沒有求導階數; 設定`dx=1`, 階導數, x軸的影像梯度
- 程式實例`ch13_2.py`: 設定`ddepth = -1`, 繪製x軸方向的影像梯度。
- PS. `ddepth=-1`, 則粗線左邊所計算出來的值為負數. 會造成資料遺失.



- `import cv2`
- `src = cv2.imread("map.jpg")`
- `dst = cv2.Sobel(src, -1, 1, 0)` # 計算 x 軸影像梯度
- `cv2.imshow("Src", src)`
- `cv2.imshow("Dst", dst)`
- `cv2.waitKey(0)`
- `cv2.destroyAllWindows()`

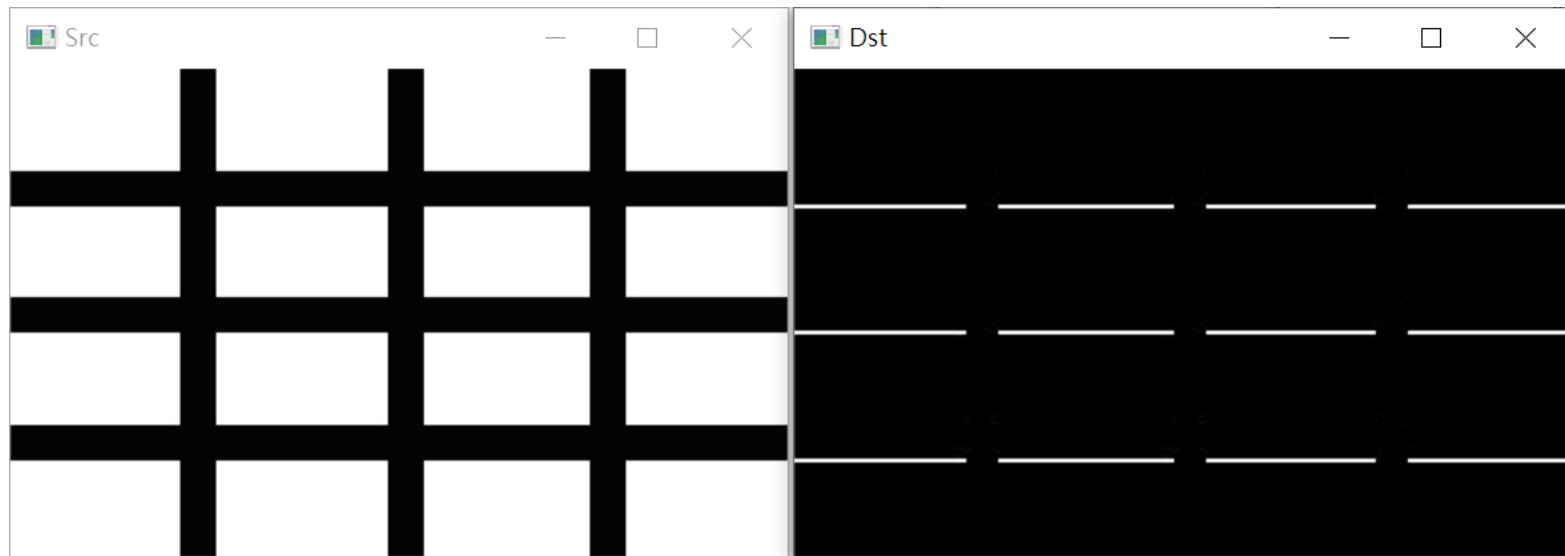
- 程式實例使用 `convertScaleAbs()` 函數將負值的梯度改為正值，讓直線的邊緣都能呈現。



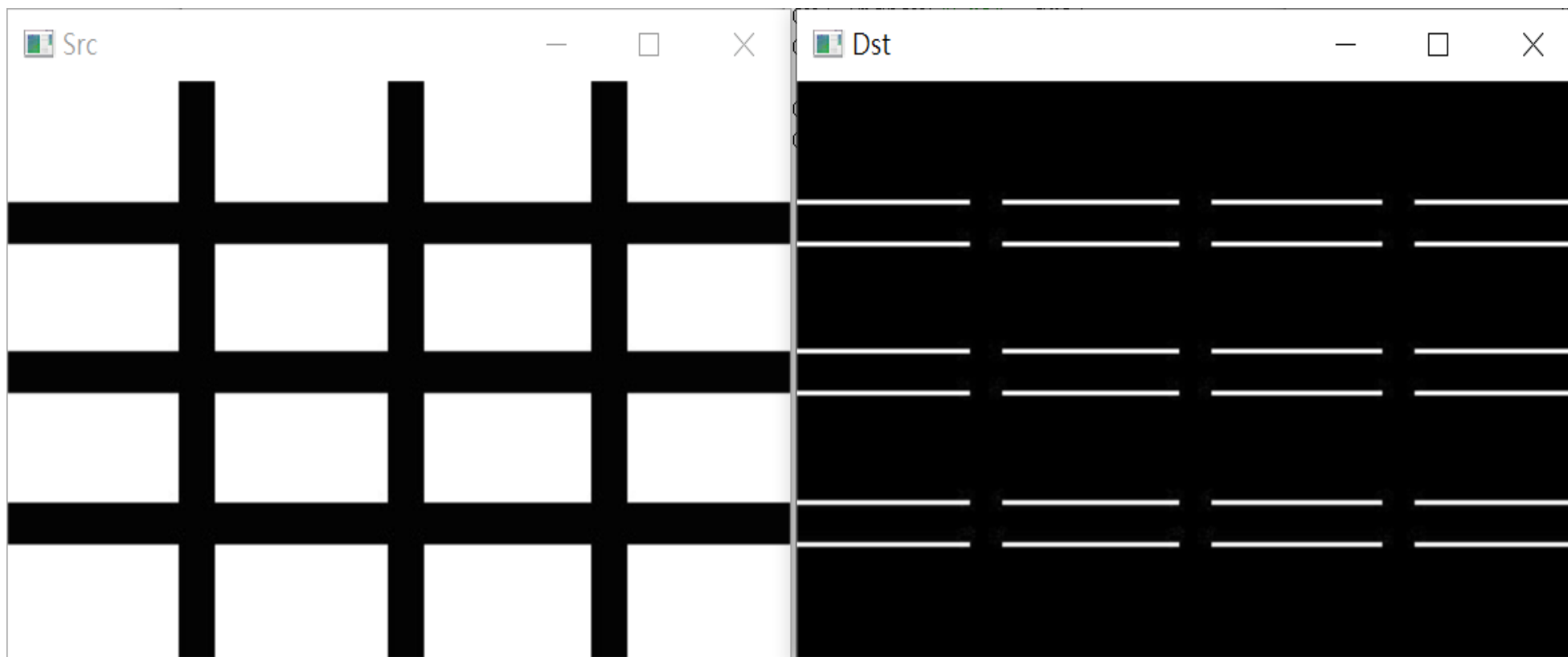
- `import cv2`
- `src = cv2.imread("map.jpg")`
- `dst = cv2.Sobel(src, cv2.CV_32F, 1, 0) # 計算 x 軸影像梯度`
- `dst = cv2.convertScaleAbs(dst) # 將負值轉正值`
- `cv2.imshow("Src", src)`
- `cv2.imshow("Dst", dst)`
- `cv2.waitKey(0)`
- `cv2.destroyAllWindows()`

13-2-7：y軸方向的影像梯度

- `dst = cv2.Sobel(src, ddepth, 0, 1)` # 設定1階導數，y軸的影像梯度
- 程式實例：設定`ddepth = -1`，繪製y軸方向的影像梯度。

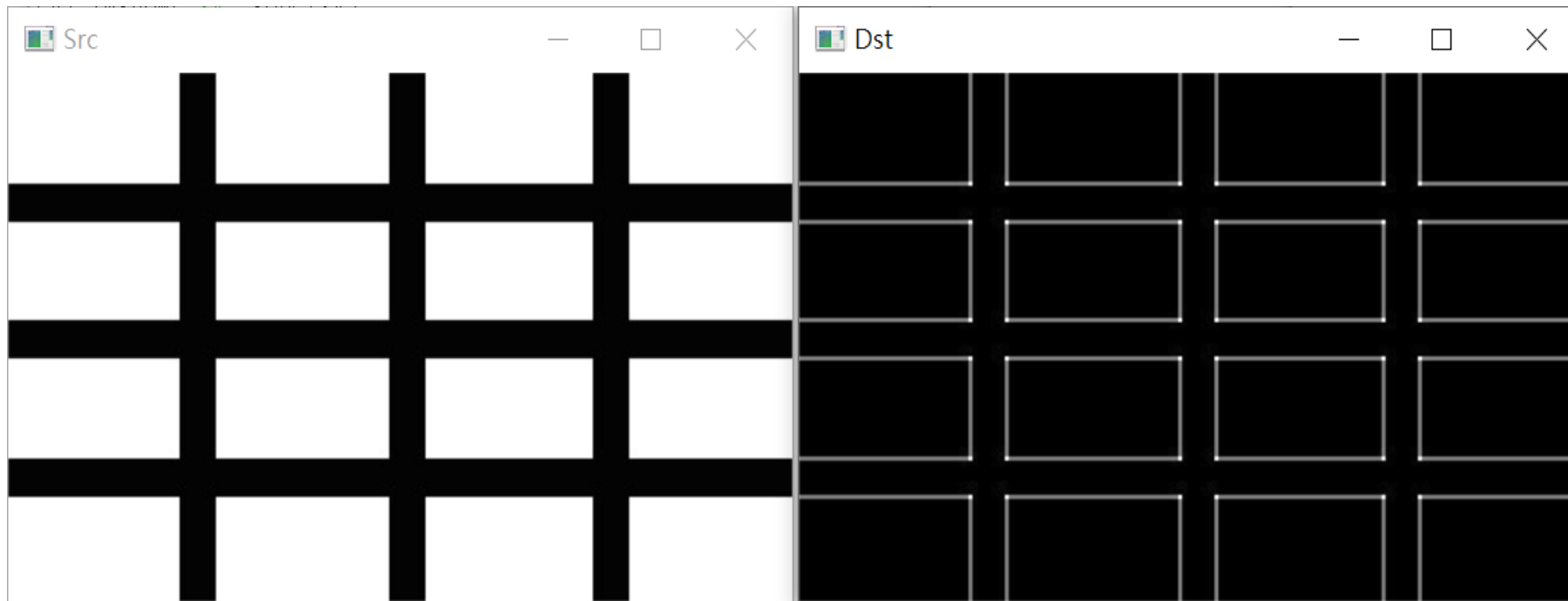


- 程式實例：使用 `convertScaleAbs()` 函數將負值的梯度改為正值，呈現出雙邊水平邊緣線。



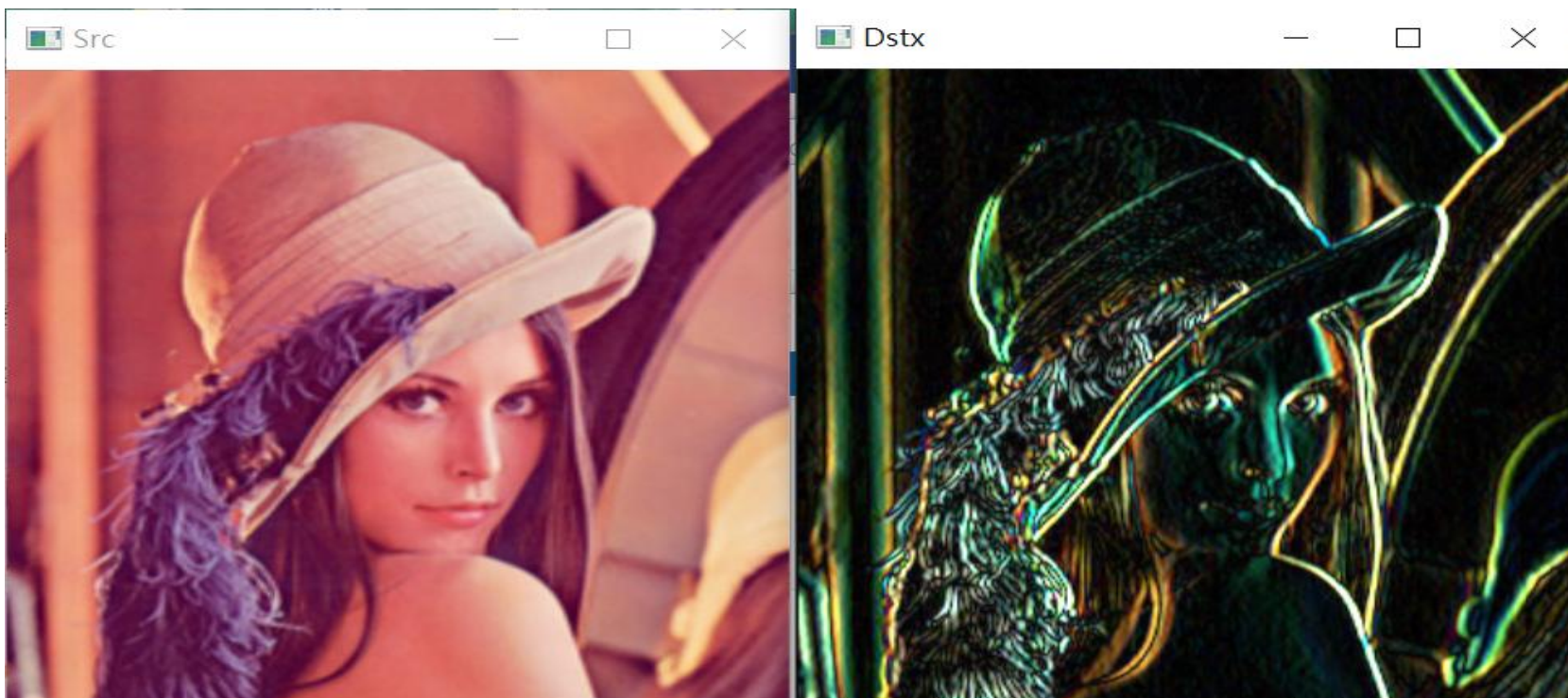
13-2-8：x軸和y軸影像梯度的融合

- 程式實例：將X梯度與Y梯度的影像融合。

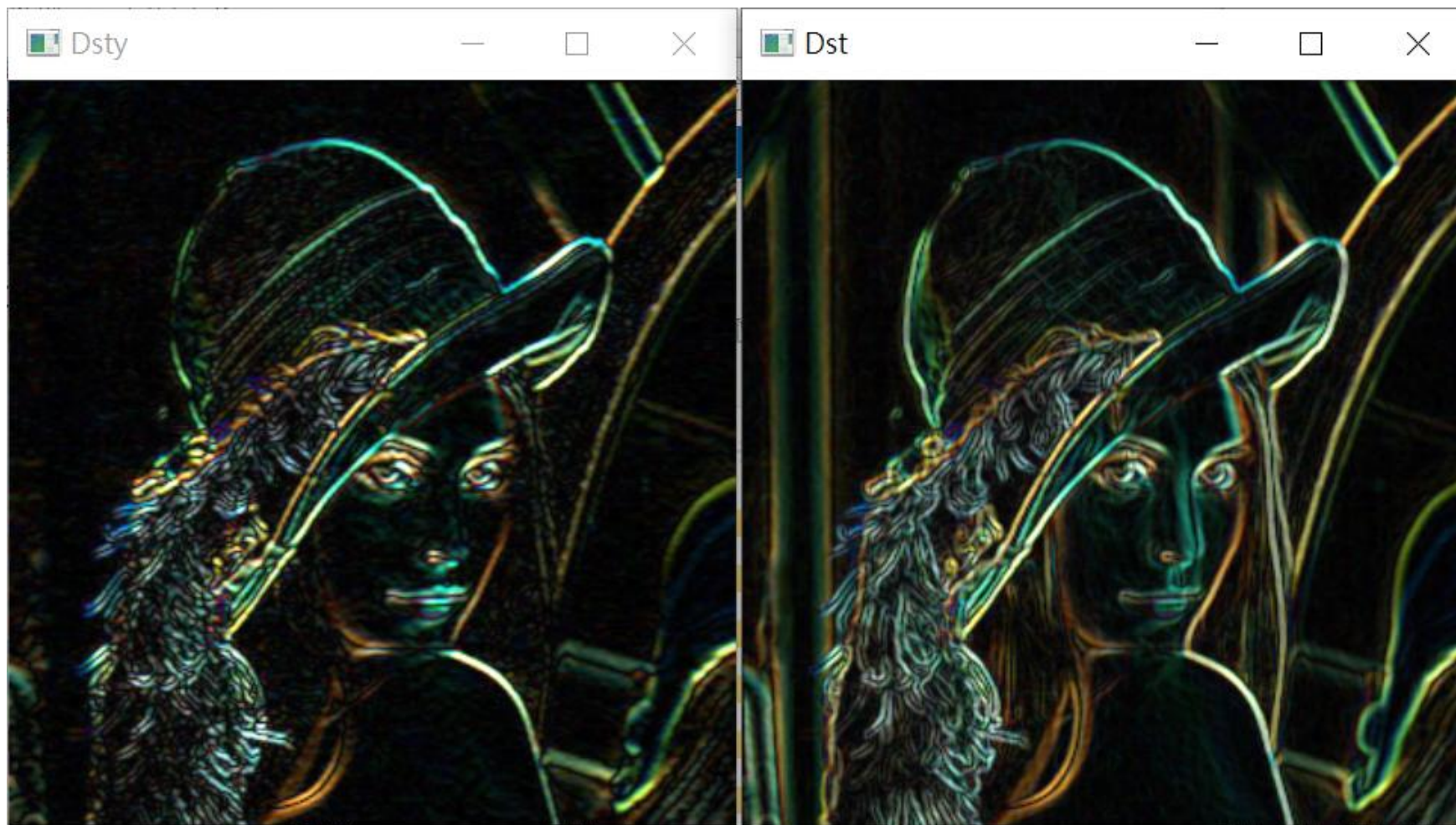


- import cv2
- src = cv2.imread("map.jpg")
- dstx = cv2.Sobel(src, cv2.CV_32F, 1, 0) # 計算 x 軸影像梯度
- dsty = cv2.Sobel(src, cv2.CV_32F, 0, 1) # 計算 y 軸影像梯度
- dstx = cv2.convertScaleAbs(dstx) # 將負值轉正值
- dsty = cv2.convertScaleAbs(dsty) # 將負值轉正值
- dst = cv2.addWeighted(dstx, 0.5, dsty, 0.5, 0) # 影像融合 等同開更號
- cv2.imshow("Src", src)
- cv2.imshow("Dst", dst)
- cv2.waitKey(0)
- cv2.destroyAllWindows()

- 程式實例：將Sobel()函數應用到實際影像，繪製原始影像、x軸影像梯度、y軸影像梯度、融合x軸和y軸的影像梯度（利用addWeightedn函數,來進行兩張影像的融合）。
- `dst = addWeighted(src1, alpha, src2, beta, gamma)`



- 下列是y軸梯度影像，x軸和y軸梯度融合的影像。



- 練習: 請讀取LENA.jpg圖檔

13-3 : OpenCV函數Scharr()

- Sobel邊緣計算的缺點對於訊號比較弱的影像邊緣效果比較不好
- Scharr將像素值之間的差異化擴大。此方法雖然可以強調更微弱的邊緣影像，但是過度強調細節，也可能會讓整個影像邊緣顯得更複雜。

- 13-3-1 : Scharr算子

$$\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

用於 x 軸算子

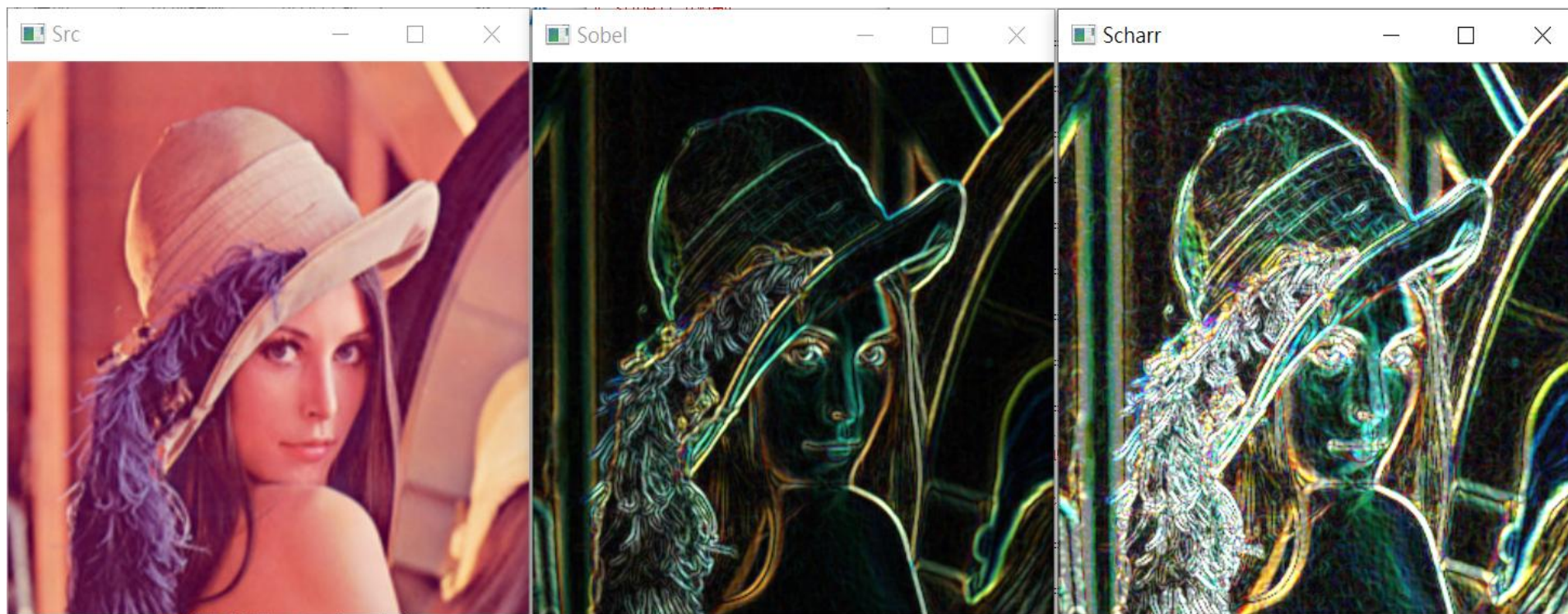
$$\begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

用於 y 軸算子

13-3-2 : Scharr()函數

- `dst = cv2.Scharr(src, ddepth, dx, dy, ksize, scale, delta, borderType)`
- **Dx:** x 軸求導階數, 一般為0,1,2, 若是0表示這個方向沒有求導階段
- **Dy:** y軸求導階數,一般為0,1,2, 若是0表示這個方向沒有求導階段

- 程式實例：更改ch13_8.py，使用彩色讀取，讀者可以比較執行結果。



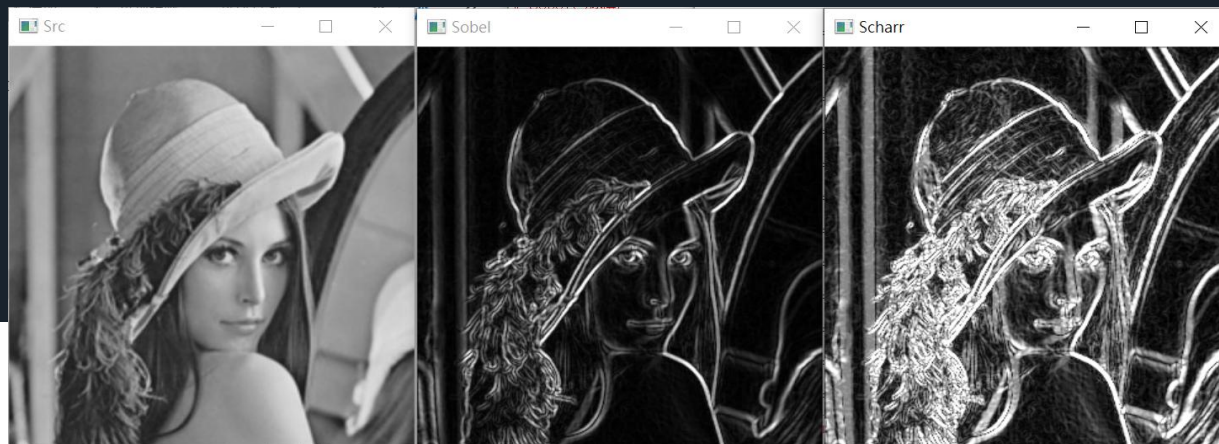
程式實例8.py：使用灰階讀取lena.jpg影像，將Sobel()和Scharr()函數所獲得的影像邊緣做比較。

```
import cv2

# Sobel()函數
src = cv2.imread("lena.jpg",cv2.IMREAD_GRAYSCALE)      # 黑白讀取
dstx = cv2.Sobel(src, cv2.CV_32F, 1, 0)                 # 計算 x 軸影像梯度
dsty = cv2.Sobel(src, cv2.CV_32F, 0, 1)                 # 計算 y 軸影像梯度
dstx = cv2.convertScaleAbs(dstx)                         # 將負值轉正值
dsty = cv2.convertScaleAbs(dsty)                         # 將負值轉正值
dst_sobel = cv2.addWeighted(dstx, 0.5,dsty, 0.5, 0)      # 影像融合
# Scharr()函數
dstx = cv2.Scharr(src, cv2.CV_32F, 1, 0)                # 計算 x 軸影像梯度
dsty = cv2.Scharr(src, cv2.CV_32F, 0, 1)                # 計算 y 軸影像梯度
dstx = cv2.convertScaleAbs(dstx)                         # 將負值轉正值
dsty = cv2.convertScaleAbs(dsty)                         # 將負值轉正值
dst_scharr = cv2.addWeighted(dstx, 0.5,dsty, 0.5, 0)     # 影像融合

# 輸出影像梯度
cv2.imshow("Src", src)
cv2.imshow("Sobel", dst_sobel)
cv2.imshow("Scharr", dst_scharr)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

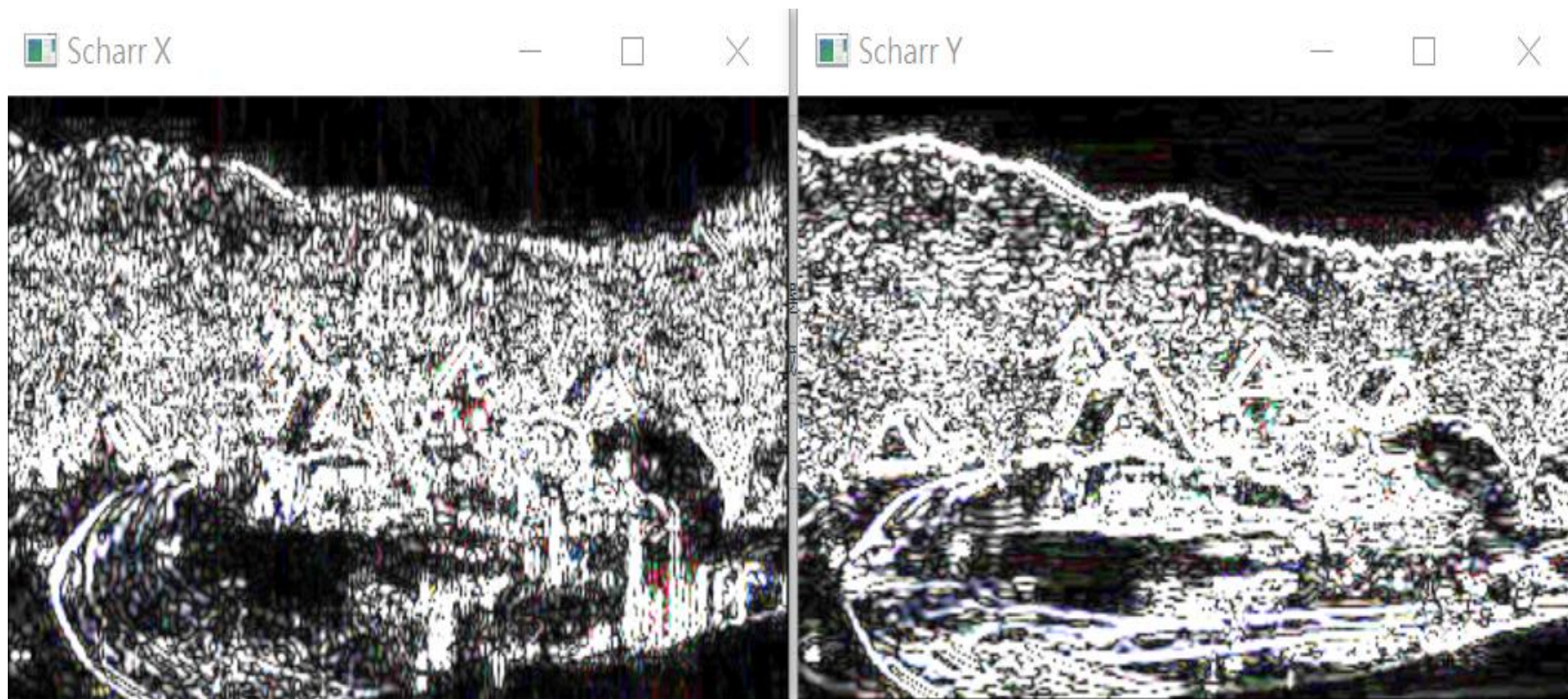


- 程式實例：原始影像是snow.jpg，請使用Scharr()函數建立此影像的x軸和y軸影像梯度，同時使用Sobel()函數和Scharr()函數建立此完整影像梯度，最後比較結果。



- import cv2
- # Sobel()函數
- src = cv2.imread("snow.jpg") # 彩色讀取
- dstx = cv2.Sobel(src, cv2.CV_32F, 1, 0) # 計算 x 軸影像梯度
- dsty = cv2.Sobel(src, cv2.CV_32F, 0, 1) # 計算 y 軸影像梯度
- dstx = cv2.convertScaleAbs(dstx) # 將負值轉正值
- dsty = cv2.convertScaleAbs(dsty) # 將負值轉正值
- dst_sobel = cv2.addWeighted(dstx, 0.5, dsty, 0.5, 0) # 影像融合
- # Scharr()函數
- dstx = cv2.Scharr(src, cv2.CV_32F, 1, 0) # 計算 x 軸影像梯度
- dsty = cv2.Scharr(src, cv2.CV_32F, 0, 1) # 計算 y 軸影像梯度
- dstx = cv2.convertScaleAbs(dstx) # 將負值轉正值
- dsty = cv2.convertScaleAbs(dsty) # 將負值轉正值
- dst_scharr = cv2.addWeighted(dstx, 0.5, dsty, 0.5, 0) # 影像融合
- # 輸出影像梯度
- cv2.imshow("Src", src) cv2.imshow("Scharr X", dstx) cv2.imshow("Scharr Y", dsty) cv2.imshow("Sobel", dst_sobel)
- cv2.imshow("Scharr", dst_scharr) (自行換行)
- cv2.waitKey(0) cv2.destroyAllWindows() (自行換行)

- 下列是Scharr()函數建立的x軸和y軸影像梯度。



- 下列左邊是Sobel()函數建立的邊緣影像，右邊是Scharr()函數建立的邊緣影像。



13-4 : OpenCV函數Laplacian()

- 影像邊緣的取得主要是將影像的邊緣資訊凸顯。所以可利用微分來達到此效果
- 13-4-1 : 二階微分
- 一階微分:

$$\frac{\partial f}{\partial x} = f(x + 1) - f(x)$$

- 一階微分如果影像像素值變化小的區域，所得到的微分結果數值也會比較小。如果變化較大的區域，微分結果數值也會較大
- 二階微分

$$\frac{\partial^2 f}{\partial x^2} = f(x + 1) + f(x - 1) - 2f(x)$$

- 二階分所獲得的影像像素值的變化率，對像素值變化小的區域沒有影響，但變化大的區域有明顯的邊緣效果。
- 所以，一階微分雖然可以得到影像的邊緣，但所獲得是較粗糙的結果；再經過二次微分後則可以得到較細微的邊緣結果。

- 13-4-2 : Laplacian算子

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

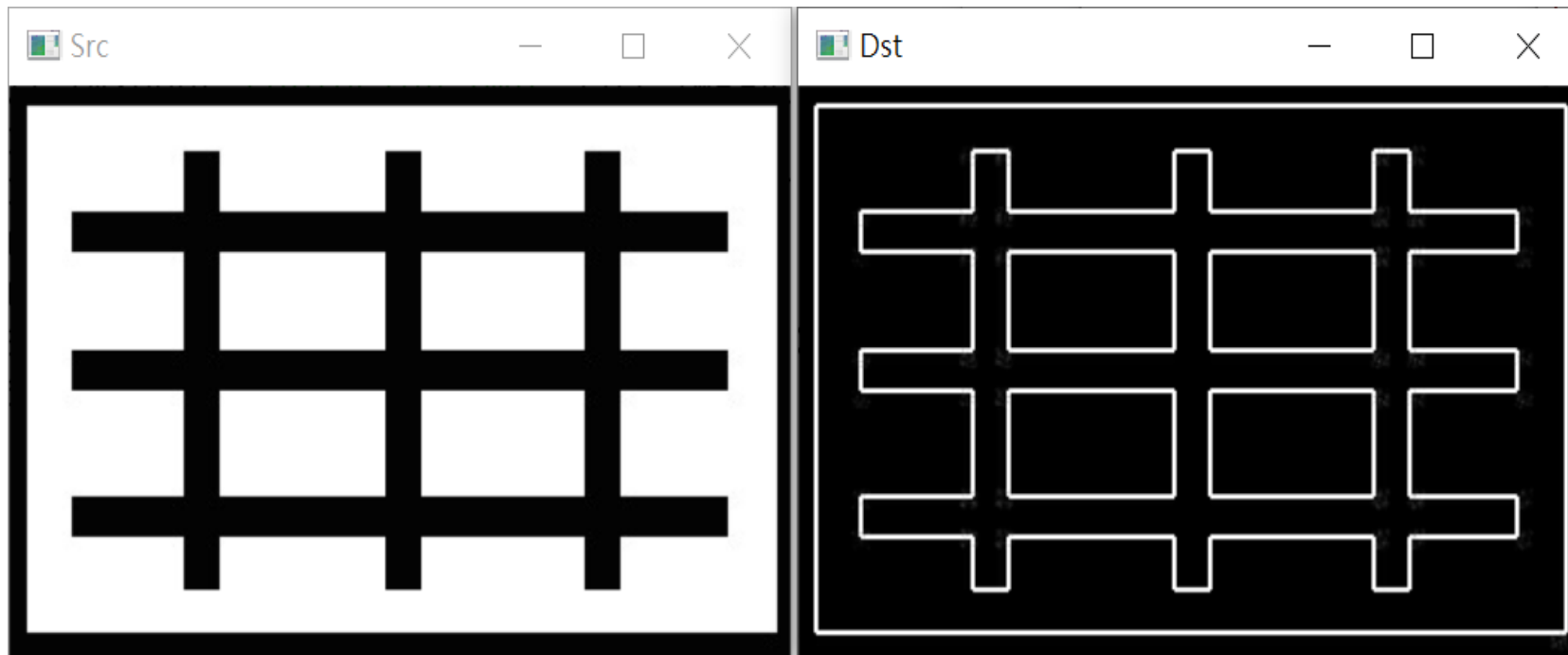
$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) - f(x, y-1) - 4f(x, y)$$

- Laplacian對應的內核

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

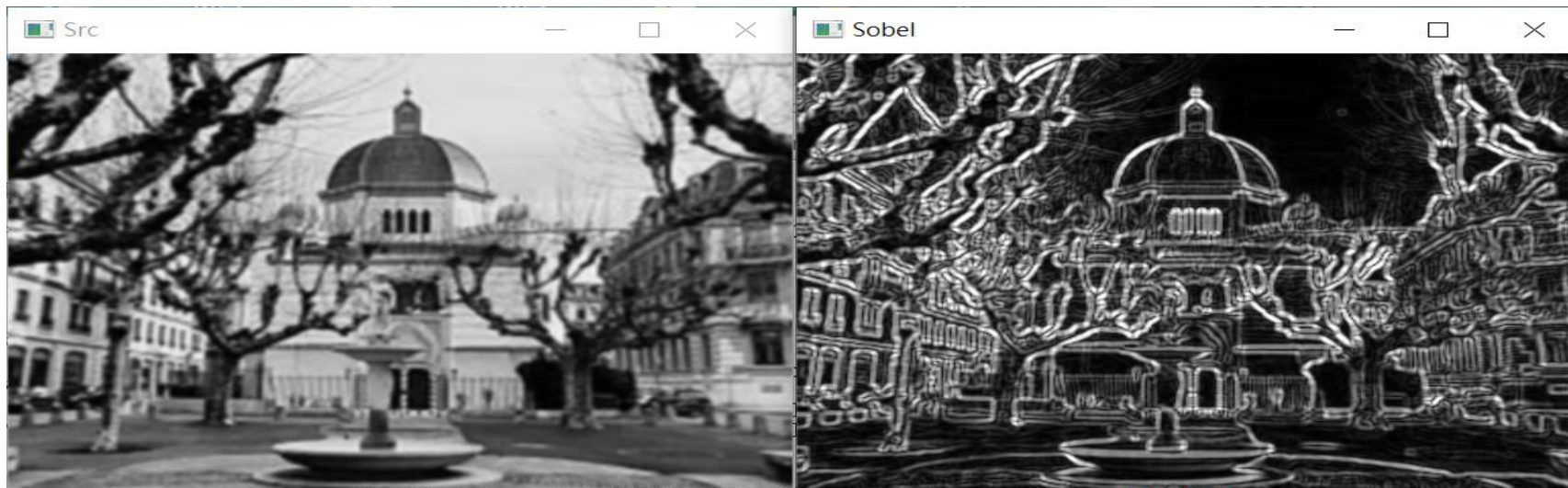
13-4-3 : Laplacian()函數

- `dst = cv2.Laplacian(src, ddepth, ksize, scale, delta, borderType)`
- `delta`: 可選參數, 表示可以加到`dst`的值
- `borderType` : 可選參數, 邊界值, 建議使用預設值即可
- 程式實例：使用`Laplacian( )`函數偵測影像邊緣的應用。



- 程式練習：瑞士日內瓦建築物的邊緣偵測，這個程式會分別列出原始影像、Sobel()、Scharr()和Laplacian()函數的執行結果。
- Ddepth:CV_32F

- 下列是原始影像和Sobel()函數的執行結果。



- 下列是Scharr()和Laplacian()函數的執行結果。



13-5 : Canny邊緣檢測

- 13-5-1 : 認識Canny邊緣檢測
- Canny不能被單獨稱為一種方法，因為它是一連串的過程加上其它方法，先模糊化去除不必要的像素、再使用類似Sobel方式取得XY軸邊緣。(模糊化就是將影像中快速變化的像素平滑化，使雜訊或細節被抹除，看起來比較平順。例如GaussianBlur(), blur(), medianBlur())
- Canny的使用相當簡單，傳入影像參數並指定兩個門檻參數threshold1與threshold2。圖形的任一點像素，若其值大於threshold2，則認定它屬於邊緣像素，若小於threshold1則不為邊緣像素，介於兩者之間則由程式依其像素強度值運算後決定。
- Canny的目標是找到一個最好的邊緣檢測法，這個方法必須有下列要件：
 - 好的檢測：演算法盡可能多地標出影像的實際邊緣。
 - 好的定位：標出的邊緣要與實際影像相符。
 - 最小響應：影像的邊緣只標出一次，同時雜訊不被標示為邊緣。

13-5-2 : Canny演算法的步驟

- 在Canny演算法的步驟分別如下：

1.降低噪音(Noise Reduction)

- Canny認為含有未經降低噪音的影像不可能會有好的邊緣結果，所以使用**高斯濾波器先降噪**。得到的影像雖然會有些模糊，但是對高斯平滑的影像部份沒有影響。

• 2：找尋影像的亮度梯度(Find Intensity Gradient of the Image)

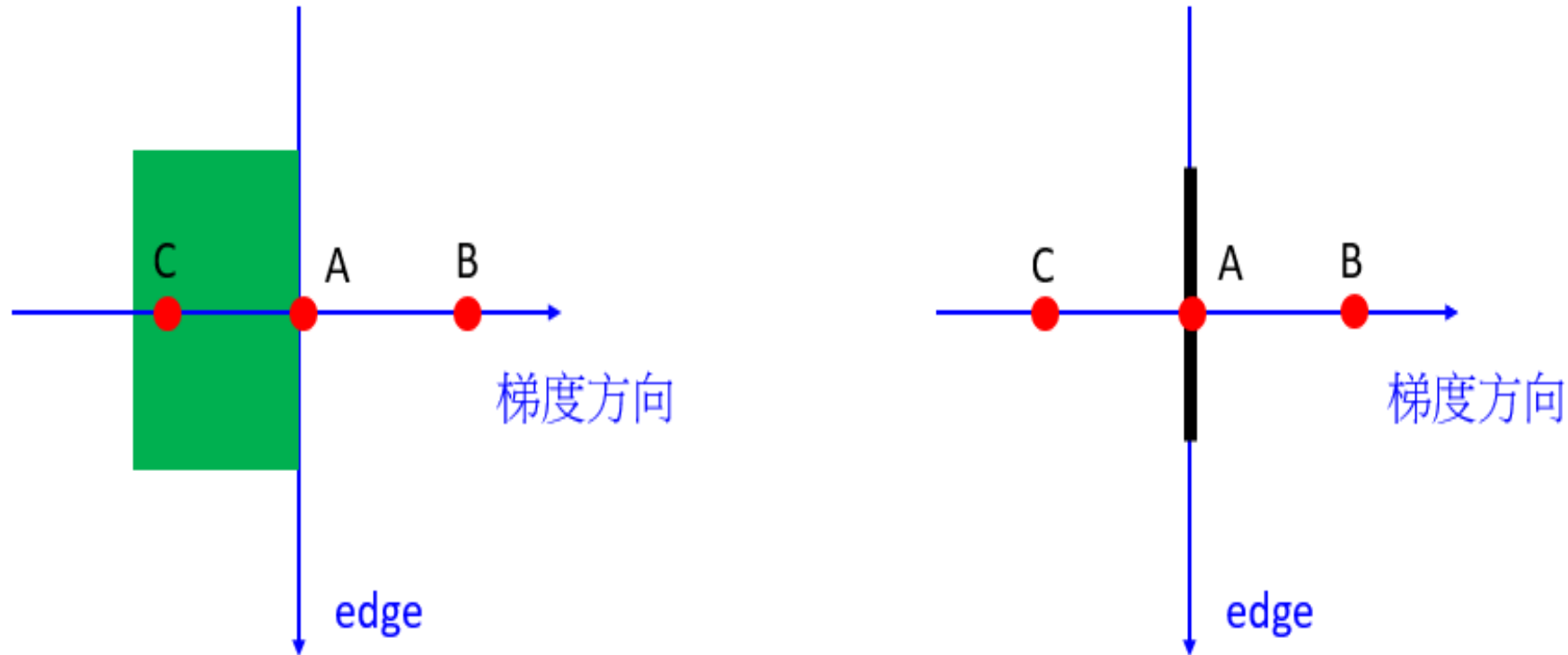
- 影像邊緣可能存在不同方向，先用Sobel處理取得 x 與 y軸的一階導數，找到每個像素點的梯度與方向 (梯度方向總是與邊緣垂直，會被就近最值為水平，垂直與雙對角線方向)

$$Edge_Gradient(G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle(\theta) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

- 3：非最大值則抑制(Non-maximum Suppression)

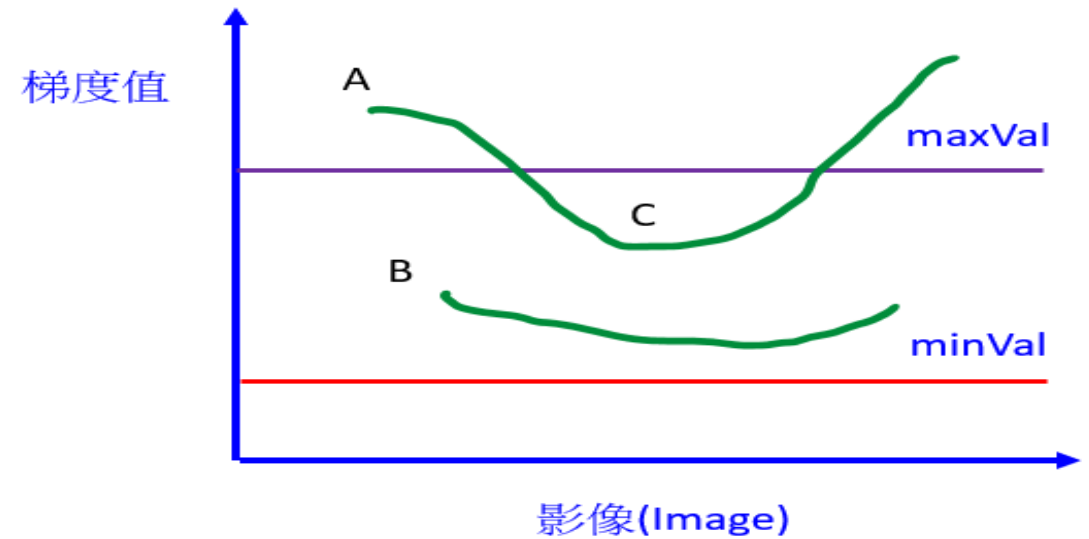
- 獲得梯度大小和方向後，將不構成的邊緣像素值拋棄。因此，檢查像素點是否在梯度方向的鄰域中是局部最大值



- A點在邊緣線上(垂直方向)，梯度方向與邊緣垂直，點B和點C在梯度方向，將A點利用點B與C來檢查是否為局部極大值，如果是則保留到下一個步驟，不是則將此點拋棄

4：滯後閾值(Hysteresis Thresholding)

- 此階段決定哪些是真正的邊緣，Canny使用兩個閾值(門檻值)，分別為高閾值(maxVal)和低閾值(minVal)
- 決定像素是否為邊緣的方式如下：
 - ❑ 如果像素點梯度值大於maxVal一定是邊緣。(紫色之上)
 - ❑ 如果像素點梯度值小於minVal一定不是邊緣，可以拋棄。(紅色之下)
 - ❑ 像素點梯度值介於maxVal和minVal之間，如果他們連接到確定邊緣像素，則被認為是邊緣，否則拋棄。



- ❑ 點A的梯度值高於maxVal, 所以確定是邊緣。
- ❑ 點B的梯度值介於maxVal和minVal之間, 可是沒有連到確定邊緣, 所以B點不是邊緣。
- ❑ 點C的梯度值介於maxVal和minVal之間, 可是有連到確定邊緣, 所以C點是邊緣。

13-5-3 : Canny()函數

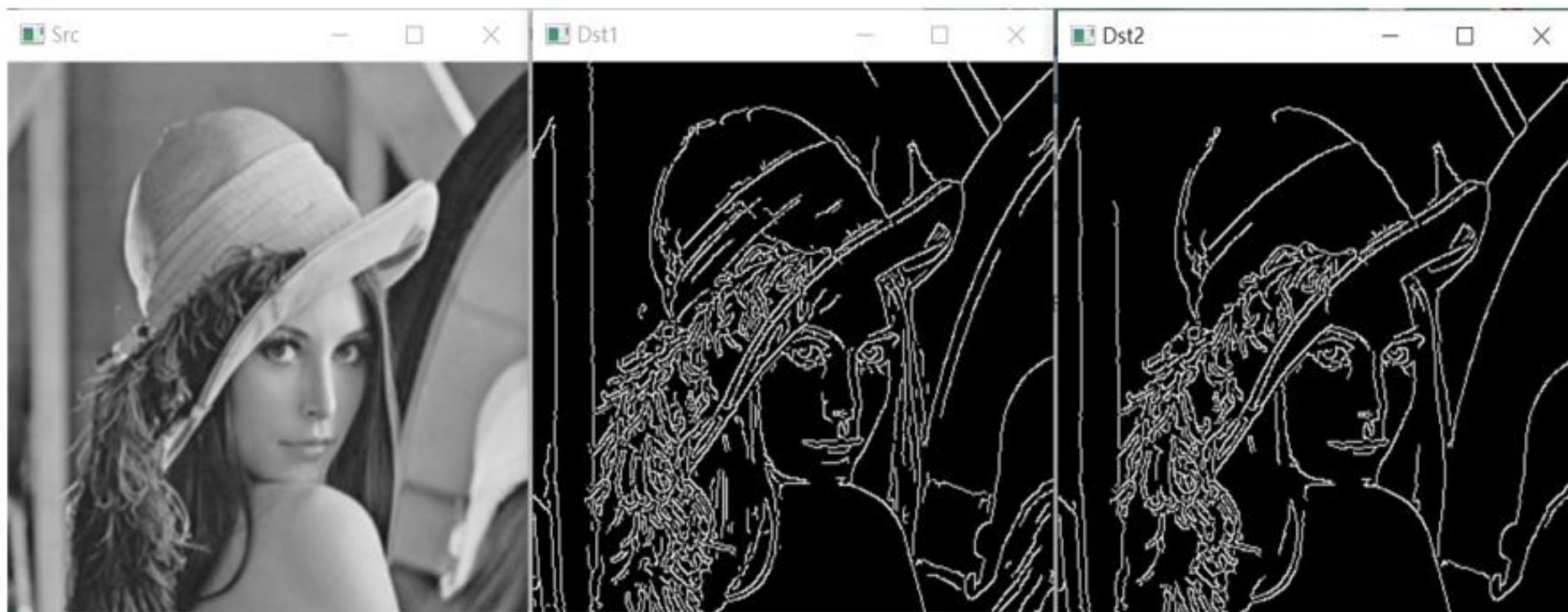
- `dst = cv2.Canny(image, edges, threshold1, threshold2, apertureSize = 3, L2gradient = False)`
- `image, edges` : 輸入和輸出的圖片。輸入必須要是灰階，而輸出結果會被 assign 到 `edges` 上面，所以 `edges` 不需要初始化。
- `threshold1, threshold2` : 用來區分 strong edge 和 weak edge，範圍都是 0 ~ 255，會在實作過程中進一步討論，通常選擇 `threshold2 / threshold1 = 1/2 ~ 1/3`，例如 (70, 140), (70, 210)
- `apertureSize` : 運算子的大小; 用來計算梯度的 kernel size，也就是 Sobel 的 `ksize`
- `L2gradient` : 可選參數，預設是 `False`。指定梯度的公式，若選擇為 `true`, 則為平方根：

$$Edge_Gradient(G) = \sqrt{G_x^2 + G_y^2}$$

- 若選擇 `false`，則梯度公式取絕對值：

$$Edge_Gradient(G) = |G_x| + |G_y|$$

- 程式實例ch13_12.py：使用minVal = 50和maxVal分別是100, 200，偵測lena.jpg的影像邊緣。

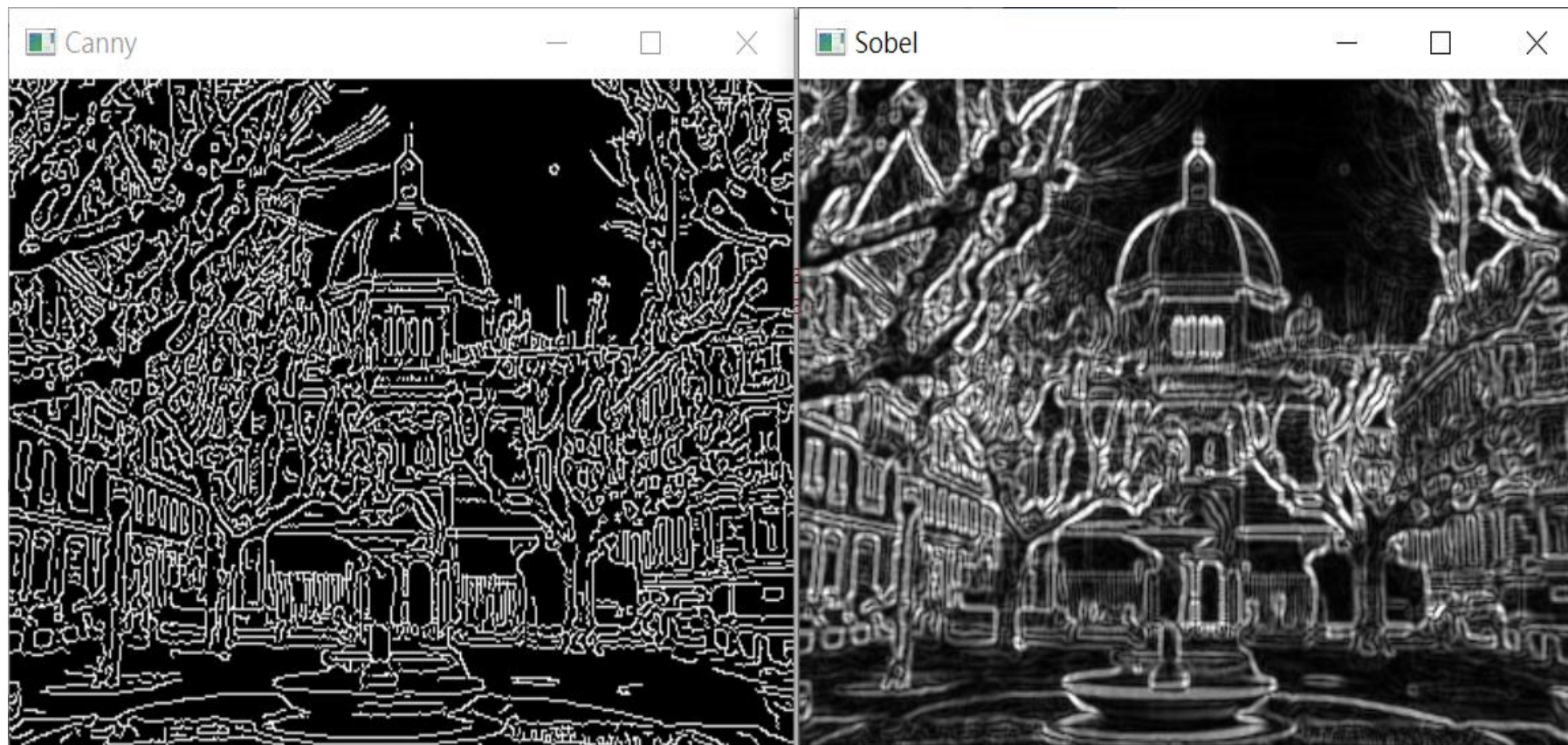


原始影像

minVal=50, maxVal=100

minVal=50, maxVal=200

- 程式實例ch13_13.py：重新設計ch13_11.py，增加使用Canny檢測方法，最後列出4種方法的結果並做比較。
- 下列左邊是Canny()和右邊是Sobel()函數的執行結果。



- 下列是Scharr()和Laplacian()函數的執行結果。



- 請讀取Baboon512圖檔, 分別使用Sobel, Canny (低閾值50, 高閾值=80), 以Laplacian(Ksize=3)及Scharr四個函數的來建立影像的邊緣。

